

University of Essex

MSc Artificial Intelligence

Module: IA_PCOM7E January 2026

Unit 7: Collaborative Discussion – Agent Communication Languages

Initial Discussion

The discussion at hand revolves around two topics, each addressed in turn:

1. What are the potential advantages and disadvantages of the use of agent communication languages such as KQML?
2. How do they compare with method invocation in Python or Java?

For the first topic, I would assert that KQML offers the advantages of an asynchronous, content-agnostic, declarative approach. Autonomous agents trade messages with each other based on intent, rather than providing specific details about implementation of that intent (Das et al, 2023). In other words, agents don't need to know about the internal workings of other agents, and communicate with each other using directives for action (e.g., "do this task") that are independent of how the task will be accomplished in terms of runtime code or data manipulation (Das et al, 2023). A significant drawback, however, is that debugging distributed agent communications can quite complex, especially given the limited tools currently available (Das et al, 2023).

For the second topic, my view is that method invocations within code take an orthogonal, if not diametrically opposite, approach, as they are usually synchronous and tightly coupled to the data they work with. Strongly typed languages such as Java, in particular, are far less flexible than KQML in that they depend on all actors within the operational runtime using the same data typing schemas in a manner compatible with Java's data processing capabilities and limitations (Lelek and Skeet, 2022).

References

Das, A., Chen, S.-C., Shyu, M.-L. and Sadiq, S. (2023). Enabling Synergistic Knowledge Sharing and Reasoning in Large Language Models with Collaborative Multi-Agents. [online] doi:<https://doi.org/10.1109/cic58953.2023.00021>.

Lelek, T. and Skeet, J. (2022). *Software Mistakes and Tradeoffs: How to Make Good Programming Decisions*. Simon and Schuster.

Peer Response No. 1

Your post offers a clear comparison between KQML and method invocation, particularly in highlighting the flexibility and abstraction provided by agent communication languages. I agree that KQML enables agents to interact based on intent rather than implementation, which supports loose coupling and autonomy in multi-agent systems (Wooldridge, 2009).

An additional aspect to consider is standardisation and interpretation. While KQML is content-agnostic, this can lead to ambiguity unless supported by shared ontologies or formal semantics. This limitation has led to the development of more standardised alternatives such as FIPA-ACL, which improves interoperability between agents (Finin et al., 1994; Bellifemine, Poggi and Rimassa, 2007).

Your comparison with method invocation is accurate, especially regarding tight coupling. However, modern approaches such as microservices and APIs have introduced more loosely coupled communication patterns, partially bridging this gap (Newman, 2021). Despite this, method invocation still lacks the semantic richness of agent communication languages.

Additionally, it is worth noting that ACLs may introduce performance overhead, making them less suitable for time-critical systems compared to direct method calls (Jennings, Sycara and Wooldridge, 1998).

References

Bellifemine, F., Poggi, A. and Rimassa, G. (2007). Developing Multi-Agent Systems with JADE. *Lecture Notes in Computer Science*. doi:<https://doi.org/10.1002/9780470058411>.

Finin, T., Fritzson, R., McKay, D. and McEntire, R. (1994). KQML as an agent communication language. *Conference on Information and Knowledge Management*, pp.456–463. doi:<https://doi.org/10.1145/191246.191322>.

Jennings, N.R., Sycara, K. and Wooldridge, M. (1998). A Roadmap of Agent Research and Development. *Autonomous Agents and Multi-Agent Systems*, 1(1), pp.7–38. doi:<https://doi.org/10.1023/a:1010090405266>.

Newman, S. (2021). *Building Microservices*. 'O'Reilly Media, Inc.'

Wooldridge, M.J. (2009). *An introduction to multiagent systems*. Chichester: John Wiley.

Peer Response No. 2

Your discussion provides a clear and well-articulated distinction between KQML and method invocation, particularly in highlighting the asynchronous, intent-driven nature of agent communication. The emphasis on abstraction from implementation is especially important, as it underpins the autonomy and flexibility of multi-agent systems. While your comparison is well developed, it slightly underemphasises the scalability and implementation challenges associated with deploying ACLs in large, real-world systems, where coordination overhead can become significant.

Building on your point regarding the complexity of debugging distributed communications, this challenge can be mitigated through the implementation of structured monitoring and validation mechanisms. For instance, incorporating message traceability systems and formal communication protocols can significantly improve transparency and fault detection in agent interactions, particularly in large-scale distributed environments (Das et al., 2023).

Additionally, while KQML's content-agnostic design enhances flexibility, it also introduces the risk of semantic ambiguity and misaligned expectations between agents. This becomes particularly significant in collaborative environments where multiple agents interact with differing objectives. Research highlights that effective coordination in such systems depends not only on communication protocols but also on trust, fairness, and explainability in decision-making processes, ensuring that interactions remain interpretable and reliable (Fornara et al., 2023).

Furthermore, recent developments in multi-agent systems demonstrate that collaborative agent architectures can significantly enhance problem-solving performance through distributed reasoning and knowledge sharing, particularly in complex and dynamic environments (Das et al., 2023). Empirical evidence indicates that such systems can achieve improved decision accuracy and quality compared to single-agent approaches, while also enhancing transparency through traceable decision processes (Kumar, 2025). However, these advantages are accompanied by notable challenges, including increased computational costs, coordination complexity, and the risk of bias amplification, which may undermine system reliability if not properly managed (Kumar, 2025).

Your comparison with method invocation could therefore be extended by considering hybrid architectures, where direct function calls support performance-critical tasks while ACLs enable adaptive, large-scale coordination across agents.

References

Fornara, N., Cheriyan Fornara, N. and Mertzani, A. (2023). Coordination, Organizations, Institutions, Norms, and Ethics for Governance of Multi-Agent

Systems XVI.

Das, A., Chen, S., Shyu, M. and Sadiq, S. (2023). Enabling synergistic knowledge sharing and reasoning in large language models with collaborative multi-agents.

Kumar, S. (2025). Multi-agent AI systems in finance: Models, applications, and challenges, *International Journal of Advanced Research in Computer Science & Technology* (IJARCST), 8(1), pp. 11555–11573.

Discussion Summary

The discussion revolved around two topics, each addressed in turn:

1. The potential advantages and disadvantages of the use of agent communication languages such as KQML;
2. A comparison of inter-agent communication languages with method invocation in Python or Java.

For the first topic, I asserted that KQML offers the advantages of an asynchronous, content-agnostic, declarative approach that focuses on the task to be accomplished over the internal workings of any given agent. I also noted that a drawback is that this approach can cause debugging agent activity to become far more complex (Das et al, 2023).

For the second topic, I asserted that method invocations are usually synchronous and tightly coupled to the data they work with, which reflects a quite different, if not practically opposite, approach to KQML, as well as one limited by the Java language's data processing capabilities (Lelek and Skeet, 2022).

Peer responses raised the salient point that KQML's declarative, command-oriented approach can result in heightened ambiguity of agent task purpose, unless supported by a shared ontology or formal semantic system for the messages exchanged between agents. (Finin et al., 1994; Kumar, 2025; Bellifemine, Poggi and Rimassa, 2007). It can also result in slower performance or greater consumption of runtime resources, as well as an increased risk of unmediated, inherent bias amplification in system outcomes (Finin et al., 1994; Kumar, 2025; Bellifemine, Poggi and Rimassa, 2007).

In conclusion, I agree with the peer responses that adoption of KQML, or any similar approach, should be tempered by an awareness of its potential drawbacks and by an active effort to incorporate solutions to those drawbacks from the start of any development project that uses it.

References

Bellifemine, F., Poggi, A. and Rimassa, G. (2007). Developing Multi-Agent Systems with JADE. *Lecture Notes in Computer Science*. doi:<https://doi.org/10.1002/9780470058411>.

Das, A., Chen, S.-C., Shyu, M.-L. and Sadiq, S. (2023). Enabling Synergistic Knowledge Sharing and Reasoning in Large Language Models with Collaborative Multi-Agents. [online] doi:<https://doi.org/10.1109/cic58953.2023.00021>.

Finin, T., Fritzson, R., McKay, D. and McEntire, R. (1994). KQML as an agent communication language. *Conference on Information and Knowledge Management*, pp.456–463. doi:<https://doi.org/10.1145/191246.191322>.

Kumar, S. (2025). Multi-agent AI systems in finance: Models, applications, and challenges, *International Journal of Advanced Research in Computer Science & Technology* (IJARCST), 8(1), pp. 11555–11573.

Lelek, T. and Skeet, J. (2022). *Software Mistakes and Tradeoffs: How to Make Good Programming Decisions*. Simon and Schuster.